

Fall Defender: Customized Faster R-CNN and YOLO Model Development for Fall Detection Using PyTorch

Enhong Liu
Georgia Institute of Technology
225 North Avenue, Atlanta, GA
eliu320@gatech.edu

Ami Kumar
Georgia Institute of Technology
225 North Avenue, Atlanta, GA
akumar741@gatech.edu

Abstract

Detecting falls presents an intriguing challenge with significant real-world applications across various domains, including public health, elderly care monitoring, workplace safety, sports injury detection, and more. However, existing methods lack the efficiency and effectiveness required for accurate real-time fall detection. Our primary objective for this project was to develop computer vision models specifically designed for detecting falls. Utilizing the PyTorch framework, we devised a series of customized object detection models based on Faster R-CNN and YOLO by modifying pre-existing layers, with the aim of tailoring the architecture for fall detection tasks. The Faster R-CNN model initially showed promise, but encountered computational and performance limitations. Thus, we turned to YOLOv5, which allowed for more extensive hyperparameter tuning, model customization, and better handling of real-world variability. Our custom YOLO model achieved significant improvements in precision, recall, and completion time compared to Faster R-CNN. Specifically, the YOLO model provided a balanced approach, with lower false positives and faster inference times, making it suitable for real-time fall detection systems. Future work for this project includes further data augmentation, addressing false positives and false negatives, and integration with sensor data. By addressing these areas, we aim to further refine and optimize the fall detection system for practical deployment, thereby contributing to enhanced safety and well-being across various sectors.

1. Introduction/Background/Motivation

By harnessing advancements in computer vision techniques and leveraging existing PyTorch frameworks, our objective was to cultivate enhanced methodologies adept at accurately identifying fall occurrences in real-time scenarios. Through this effort, we envisioned our work to mitigate

the risk of injuries and enhance response times, ultimately safeguarding individuals from harm and contributing to the preservation of lives.

While there is growing research in fall detection, there is no significant takeoff of any universally accepted, commercial product. Common approaches involve using wearable, particle filter algorithms to detect drastic speed changes, or computer vision based analysis on single video frames (Wang et al., 2020 [2]; Gutierrez et al., 2021 [1]). However, these algorithms may produce false positives or fail to detect falls accurately in certain scenarios, such as when the fall occurs gradually or is not accompanied by significant acceleration. One successful, recent approach used the codebook background subtraction method to extract human body silhouettes, which were subsequently classified into different human postures (Yu et al., 2017 [3]). However, this approach is constrained by computational complexity and thus not feasible to deploy in real-time fall detection.

The challenge of real-time fall detection presents significant implications across various sectors, including public health, elderly care, workplace safety, and sports injury prevention. Successfully addressing this challenge holds the potential to revolutionize safety protocols and, crucially, mitigate risks associated with falls. Therefore, addressing this challenge is not merely a technological pursuit but also presents fundamental and profound humanitarian consequences.

The data utilized for our project was sourced from the Roboflow Universe project for fall detection. The dataset was created to facilitate the development and evaluation of fall detection algorithms, aiming to address the critical need for more reliable and accurate methods in real-world settings. The dataset comprises annotated images representing various scenarios where falls may occur, such as indoor and outdoor environments, workplaces, and residential settings. Each instance is annotated to indicate the presence or absence of a fall event. The dataset contains a total of over 4,500 instances, including 3,000 training data points, 900 validation data points, and 500 test data points. The data

associated with each instance was acquired through various sources, including publicly available websites and online databases scraped by the research team. Within this project, minimal preprocessing was performed on the data, limited to resizing images and standardizing annotations for consistency.

2. Approach

The project began by exploring the utility of Faster R-CNN for fall detection. Within the Faster R-CNN model, training, validation, and test datasets were consolidated into a singular dataset, subsequently partitioned randomly into training and test subsets with an 80/20 ratio. This effort aimed to maximize the utilization of available data points for both training and evaluation purposes, since the validation process is likely not required during the Faster R-CNN model training process. Following dataset preparation, we employed the object detection framework provided by PyTorch, leveraging the `torch.nn` module. The framework of PyTorch object detection can be found through this [link](#). This framework underwent customization to cater specifically to the task of fall detection by adapting the final layers to suit the nuances of fall detection. An initial step was conducted with the utilization of the Faster R-CNN architecture with ResNet50 as the backbone feature extractor. For feature extraction, the backbone was deliberately frozen to prevent superfluous updates during training, while the final fully connected layer was refined for binary classification into fall and background categories. During the training phase, input data comprising images, box coordinates, and corresponding labels were fed into the model. However, due to the high false positive rate, which will be discussed in the Experiment section, we decided to explore YOLO to enhance the performance of our model.

We also created an alternative model using YOLOv5 as a basis framework with a 88/8/4 split between training, validation, and test data. We trained this model with an SGD optimizer. We built a custom model by configuring the anchors, backbone, and head. We also finely tuned extensive hyperparameters to tailor the model to the fall detection task. Fall detection involves detecting instances where a person has fallen, and requires a sensitivity to human shapes and postures, as well as the ability to distinguish falls from a variety of backgrounds. Given the real world scenario of near real-time fall detection systems such as elder care monitoring, the priorities are speed and the presence of a fall over the exact accuracy of the bounding box around the fall. These were the constraints we kept in mind when building the YOLOv5 model. In the model, we tuned the anchors, backbone, and head to accommodate the fall detection task. We tuned the anchors to match the typical aspect ratios and sizes of humans in postures like standing, sitting, and lying down. We created a backbone comprised of a focus

layer, successive downsampling through a series of Conv and C3 layers, and a spatial pyramid pooling (SPP) layer at the end. The focus layer reduces dimensionality while preserving important features, optimizing for speed without loss of important information about posture. The alternating Conv and C3 layers increase the depth of the network and enhance the model's ability to handle complex features. Finally, the SPP layer enhances the receptive field to allow the network to capture spatial hierarchies better. We configured the head to include multiple upsampling and concatenations to ensure the model retains fine-grained details from earlier layers and accurately localizes and classifies fall incidents within diverse environments.

Before the project began, several challenges were anticipated, including: 1) The imbalance between fall and non-fall instances in the dataset could pose challenges during model training, potentially leading to biased predictions and reduced overall performance; 2) Implementing and fine-tuning complex deep learning models for fall detection requires expertise in model architecture design and hyperparameter optimization; 3) Real-world variability, such as variations in lighting conditions, camera angles, and environmental settings, could introduce much noise and thus reduce model robustness and generalization ability of the fall detection system. However, during the process of building and training the Faster R-CNN model, limitations in storage space for the 5000-image dataset and computation resources emerged as unexpected obstacles, slowing down the progress significantly. The GCP credits provided by the course were swiftly depleted after four rounds of training with Faster R-CNN, each comprising 15 epochs. Although Kaggle offered 30 hours of free GPU training, the available storage space proved inadequate to accommodate all the images required for the training process, leading to training in the Kaggle kernel not feasible as well. To address this issue, the personnel tasked with Faster R-CNN opted for Google Colab Pro service, enabling it to train the Faster R-CNN model eventually. Besides its computational complexity, the Faster R-CNN model encountered unexpectedly high loss values, even following extensive hyperparameter tuning, which will be discussed in the next section. This high loss could stem from the inherent variability in lighting, conditions, and poses within the datasets. Unfortunately, Faster R-CNN lacks the ability to explicitly tune hyperparameters to mitigate these challenges, thus we turned to the YOLO model for its inherent capability for hyperparameter tuning and extensive model customization to address these issues.

To address the challenges of variability, we explicitly tuned parameters in our YOLO model that introduced image augmentations like different hue, saturation, and value or translation, scale, and mirroring. These helped us achieve lower loss values and higher precision overall in the final

model. Building and tuning our custom model with YOLO presented its own challenges, including navigating the vast array of possible model configurations and hyperparameters to tune. Making sure the dimensions of each layer aligned was a challenge which required research of how each layer manipulated image data and some trial and error. Balancing the depth of the network, to accommodate the complexity of the features to be extracted, with the speed of detection also required a significant amount of time. We experimented with deep (15 layer) and shallow (5 layer) networks, landing on 10 layers in the backbone.

3. Experiments and Results

Within the Faster R-CNN framework, the loss values from both training and test data were monitored during the training process, while accuracy, precision, and recall rate are calculated after training for performance evaluation. For the Faster R-CNN model, hyperparameters such as the number of epochs (15), learning rate (initially set at 0.001), momentum (0.9), and weight decay (0.0005) were initiated at a reasonable number and then meticulously tuned to facilitate effective model learning. The training process yielded promising results, with a discernible decrease in loss over successive epochs (Figure 1), indicative of the model's learning trajectory. Concurrently, we also monitored the test loss throughout the training process, observing a decreasing trend followed by a plateau post approximately 10 epochs. To optimize the approach, iterative adjustments were made to key hyperparameters, including fine-tuning the learning rate and momentum to mitigate the potential overfitting issue. The results from the Faster R-CNN model are reasonable, with a final loss value of approximately 14.5. Although this loss is quite high, we observe a decreasing trend in loss during the training process. The final accuracy is 0.965, precision is 0.335, and recall rate is 0.965. These results indicate that the model can correctly identify a large portion of instances of falling down, but it also incorrectly labels many irrelevant instances, leading to a high false positive rate. An example of fall identification from the test data is shown in Figure 2, where the model correctly identifies the fall instance. The trade-off between not being able to identify a fall and falsely identifying a fall is debatable. In the real world, not detecting a fall might be more costly than falsely identifying one. However, the high false positive rate raises concerns, including the cost of resources and time in dealing with falsely identified fall instances. To further improve the utility of the fall detection model, we decided to turn to YOLO for its enhanced capability in dealing with variability.

We tuned the hyperparameters extensively on our custom YOLO model as well to optimize the fall detection task. We landed on a moderate learning rate of 0.01 which we decayed over time to fine tune and converge the weights. We

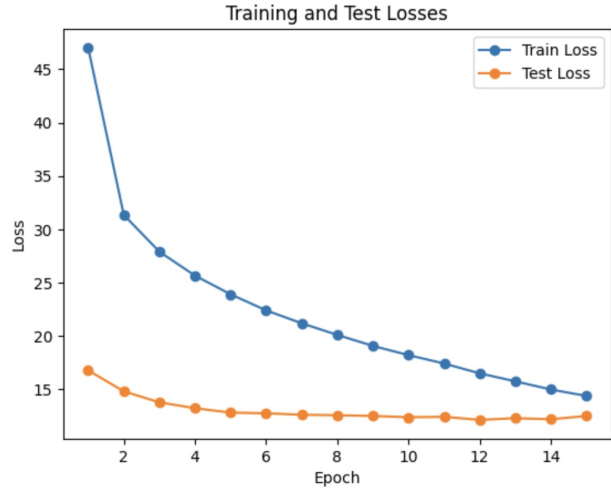


Figure 1. Training and test loss from Faster R-CNN model.



Figure 2. An example of inference results from Faster R-CNN model.

also tuned warm-up epochs and momentum to 3 and 0.8 respectively. We also weighted the box loss (box_loss), class loss (cls_loss), and objectness loss (obj_loss). YOLO uses a combination of the three loss values while training the model. The box loss measures how accurately the model predicts the bounding box coordinates, which we down-weighted since exact precision in this category is not necessary for any real world fall detection system scenarios. The objectness loss measures how well the model predicts the presence of a fall, which is the most important component of the loss in our scenario. So, we increased the weighting of this loss considerably. Finally, we introduced non-zero values for several augmentation parameters to allow HSV adjustments, random flips, translations, and scale. We experimented with three values between the given range, 0 to 1, for each and selected values with the highest precision and recall numbers in the end. These parameters help the

model generalize across different lighting conditions, angles, and orientations of falls.

We had three main measures of success for our custom YOLO model: loss (obj_loss, cls_loss, and box_loss), precision, and completion time. The results of our final YOLO model are visible in Figure 3. Our model achieved box_loss and obj_loss below 0.02 and 0.03 respectively for the training set. The box_loss converged at 0.03 and the obj_loss dipped to 0.018 on the validation set. There is evidence of overfitting past 120 epochs, where the validation obj_loss increases while the training obj_loss continues to decrease. The cls_loss is consistently 0 because there is no need for the model to differentiate between classes. There is only one class with the way this problem is set up.

The precision and recall of the custom YOLO model both converged to 0.8. This model has a more balanced accuracy than the Faster R-CNN version and provides compromises between identifying positive cases and maintaining accuracy in predictions. It avoids too many false positives and would avoid falsely alerting in fall detection systems. However, a high accuracy like that of the Faster R-CNN model is a necessary feature in use cases where there are critical detection needs and where the cost of a false negative is very high.

The mAP_0.5 value converges 0.8, while mAP_0.5:0.95 converges to a much lower value of 0.4. This means the model detects objects reliably when moderate accuracy is required, but that the models' ability to precisely localize objects is weaker. The model is good at telling roughly where objects are but is not tightly bounding exact contours of objects, which is okay for the real world use case of detecting falls. Properly detecting the presence of a fall is more important than getting the bounding box exactly right.

In terms of completion time, the YOLO model took 2.180 hours to train. The speed for detections on the test set was 0.5 ms per image for pre-processing, 7.1 ms per image for inference, and 2.0 ms per image for non-maximum suppression (post-processing). This means the model works incredibly quickly. This, combined with the high mAP_0.5 value means the model can quickly tell if a fall occurred or not and is suitable for fall detection alerting systems.

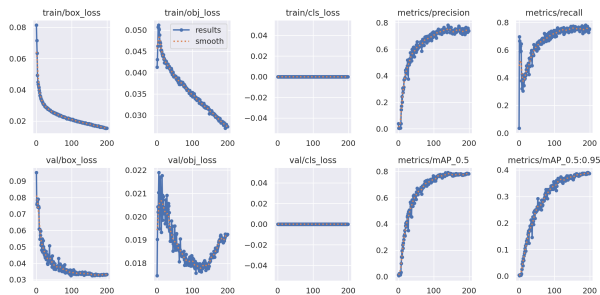


Figure 3. Results from training and validation of the custom YOLO model.



Figure 4. Seated fall that YOLO model still confidently detects as a fall.

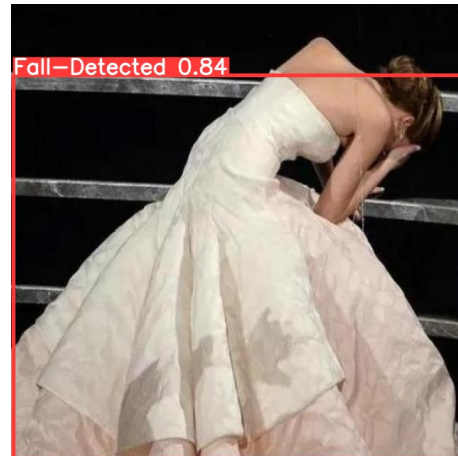


Figure 5. Upright fall that YOLO model accurately detects as a fall.



Figure 6. Some false positives as runners jump over hurdles.

Qualitatively examining fall detection on a held out test



Figure 7. False negative in an image with noise.

set shows the model is robust to a lot of unique poses. Figure 4 depicts a child who has fallen but is still sitting upright. The model is able to accurately detect this as a fall with confidence 0.81. Figure 5 shows an actress who has fallen on some stairs but appears in the image to look upright. The model is still able to discern a fall with confidence 0.84. Meanwhile, some hurdlers in figure 6 have also been detected as fallen, showing some examples of false positives. False negatives are also detected as well, as in figure 7, where noise in the image stops the model from accurately detecting the fall.

4. Discussion

The problem involved developing a fall detection system using deep learning techniques with images/video frames, which required analyzing spatial and temporal features to distinguish between fall and non-fall events. The model was developed by employing deep learning architectures including Faster R-CNN and YOLO, which are specifically designed for object detection tasks. These architectures allowed the model to learn hierarchical representations of features from input data to make predictions about fall events. Convolutional layers within the model learning parameters to extract relevant features from input images were not learned from this dataset, rather from pre-trained model to reduce computation burden. Classifying probabilities into fall or non-fall decisions involved learning parameters from the datasets in this project.

Future work for this project could include: 1) Investigating strategies to reduce false positives while maintaining high recall is crucial for practical fall detection systems, which could involve refining the decision threshold, exploring different loss functions, or incorporating contextual information to improve the model's understanding of fall events; 2) Although the current dataset covers a variety of scenarios, further augmentation techniques could be ex-

plored to simulate more diverse real-world conditions, such as different weather and camera angles; 3) While the current approach focuses on visual data, integrating additional sensor data, such as accelerometer or pressure sensor data from wearable devices, could provide complementary information for more accurate and reliable fall detection, especially in scenarios where visual cues are limited; and 4) Designing user-friendly interfaces and integrating the fall detection system into existing platforms or devices can improve accessibility and adoption, which could involve considering the needs and preferences of end-users, such as caregivers, healthcare professionals, or individuals themselves.

5. Work Division

Please add a section on the delegation of work among team members at the end of the report, in the form of a table and paragraph description. This and references do **NOT** count towards your page limit. An example has been provided in Table 1.

Student Name	Contributed Aspects	Details
Enhong Liu	Data Sourcing, Faster R-CNN Model Training, Report Writing	1. Looked for the image datasets for the project; 2. Fine-tuned Faster R-CNN model to tailor the existing framework for fall detection; 3. Drafted and polished final report
Ami Kumar	Data Sourcing, YOLO Model Training, Report Writing	1. Looked for the image datasets for the project; 2. Fine-tuned YOLO model to tailor the existing framework for fall detection; 3. Drafted and polished final report

Table 1. Contributions of team members.

References

- [1] Jesús Gutiérrez, Víctor Rodríguez, and Sergio Martín. Comprehensive review of vision-based fall detection systems. *Sensors*, 21(3), 2021. [1](#)
- [2] George Azzopardi Xueyi Wang, Joshua Ellul. Elderly fall detection systems: A literature survey. *Frontiers in Robotics and AI*, 7, June 2020. [1](#)
- [3] Miao Yu, Liyun Gong, and Stefanos Kollias. Computer vision based fall detection by a convolutional neural network. pages 416–420, 11 2017. [1](#)